

Homework 3: Transformers, T5, Machine Translation

This homework guides you through implementing a complete T5 (Text-To-Text Transfer Transformer) encoder-decoder architecture from scratch for machine translation. You will build the encoder stack, decoder stack with cross-attention, and train the model on English-German translation.

Total Points: 20

Instructions:

1. Complete all tasks in this notebook
2. Ensure your code runs without errors
3. Submit both this notebook and any additional files created
4. Write clear explanations for your approach
5. Train your model on the English-German translation dataset

Setup

```
1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4 import numpy as np
5 import matplotlib.pyplot as plt
6 import math
7 from typing import Optional, Tuple, List, Dict
8
9 # Set random seeds
10 torch.manual_seed(42)
11 np.random.seed(42)
12
13 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
14 print(f'Using device: {device}')
```

Using device: cuda

Dataset Preparation

We'll use the WMT English-German translation dataset. This dataset contains parallel sentences in English and German, perfect for training a sequence-to-sequence translation model.

```
1 !pip install -q datasets
```

```
1 import os
2 from collections import Counter
3 from datasets import load_dataset
4 import tqdm
5
6 print("Initializing WMT Dataset Loader (No backups, real data)...")
7
8 # 1. Load the Real WMT Dataset
9 # We use WMT16 (German-English), a standard benchmark.
10 # We take the first 50,000 examples to keep RAM usage efficient while ensuring variety.
11 # If you want the full dataset (4.5M pairs), remove the [:50000] slice.
12 dataset = load_dataset("wmt16", "de-en", split="train[:50000]")
13
14 print(f"Successfully loaded {len(dataset)} pairs from WMT16.")
15
16 # 2. Extract pairs into memory
17 translation_pairs = []
18 for item in dataset:
19     # WMT data structure is usually {'translation': {'de': '...', 'en': '...'}}
20     en_text = item['translation']['en']
21     de_text = item['translation']['de']
22     translation_pairs.append((en_text, de_text))
23
24 # 3. Build Vocabulary (Character-level)
25 # We stick to character-level to match your previous logic.
26 # Note: For SOTA results on WMT, you would usually use BPE (Byte Pair Encoding),
27 # but character-level is robust for learning deep learning fundamentals.
28 print("Building vocabulary from dataset...")
29
30 counter = Counter()
```

```

30 counter = Counter()
31 for en, de in translation_pairs:
32     counter.update(en) # Keep case sensitivity? standard is usually to keep it for WMT
33     counter.update(de)
34
35 # Filter rare characters to keep vocab clean (optional, but good for real web data)
36 MIN_FREQ = 5
37 common_chars = [char for char, count in counter.items() if count >= MIN_FREQ]
38
39 # Add special tokens
40 special_tokens = ['<pad>', '<sos>', '<eos>', '<unk>']
41 chars = special_tokens + sorted(common_chars)
42 vocab_size = len(chars)
43
44 print(f'Shared vocabulary size: {vocab_size}')
45
46 # 4. Mappings
47 stoi = {ch: i for i, ch in enumerate(chars)}
48 itos = {i: ch for i, ch in enumerate(chars)}
49
50 PAD_IDX = stoi['<pad>']
51 SOS_IDX = stoi['<sos>']
52 EOS_IDX = stoi['<eos>']
53 UNK_IDX = stoi['<unk>']
54
55 # 5. Encoding/Decoding Functions
56 def encode(s):
57     """Encode string to token IDs (Character level)."""
58     # We do NOT lower() here to preserve information, unlike the toy example
59     return [stoi.get(c, UNK_IDX) for c in s]
60
61 def decode(l):
62     """Decode token IDs to string."""
63     return ''.join([itos[i] for i in l if i != PAD_IDX])
64
65 print(f"Special tokens: PAD={PAD_IDX}, SOS={SOS_IDX}, EOS={EOS_IDX}, UNK={UNK_IDX}")
66
67 # 6. Train/Val Split (80/20)
68 n_train = int(0.8 * len(translation_pairs))
69 train_pairs = translation_pairs[:n_train]
70 val_pairs = translation_pairs[n_train:]
71
72 print("-" * 40)
73 print(f"Train pairs: {len(train_pairs)}")
74 print(f"Val pairs: {len(val_pairs)}")
75 print("-" * 40)
76 print("Sample Data (Real WMT):")
77 print(f"  EN: {train_pairs[5][0]}") # Random index to show real sentences
78 print(f"  DE: {train_pairs[5][1]}")
79 print(f"  Encoded: {encode(train_pairs[5][0])}")

```

```

Initializing WMT Dataset Loader (No backups, real data)...
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
README.md:      11.1k/? [00:00<00:00, 200kB/s]

de-en/train-00000-of-00003.parquet: 100%                282M/282M [00:02<00:00, 190MB/s]

de-en/train-00001-of-00003.parquet: 100%                267M/267M [00:06<00:00, 30.6MB/s]

de-en/train-00002-of-00003.parquet: 100%                277M/277M [00:04<00:00, 53.5MB/s]

de-en/validation-00000-of-00001.parquet: 100%           343k/343k [00:00<00:00, 721kB/s]

de-en/test-00000-of-00001.parquet: 100%                 475k/475k [00:00<00:00, 1.61MB/s]

Generating train split: 100%                            4548885/4548885 [00:10<00:00, 401733.89 examples/s]

Generating validation split: 100%                       2169/2169 [00:00<00:00, 62530.55 examples/s]

Generating test split: 100%                             2999/2999 [00:00<00:00, 111549.06 examples/s]

Successfully loaded 50000 pairs from WMT16.
Building vocabulary from dataset...
Shared vocabulary size: 113
Special tokens: PAD=0, SOS=1, EOS=2, UNK=3
-----
Train pairs: 40000
Val pairs:   10000
-----
Sample Data (Real WMT):
EN: Please rise, then, for this minute' s silence.
DE: Ich bitte Sie, sich zu einer Schweigeminute zu erheben.
Encoded: [45, 69, 62, 58, 76, 62, 4, 75, 66, 76, 62, 13, 4, 77, 65, 62, 71, 13, 4, 63, 72, 75, 4, 77, 65, 66, 76, 4, 70, 6

```

✓ TASK 1: T5 Encoder Implementation (6 points)

Build the encoder stack that processes source sequences with bidirectional attention. The encoder is similar to BERT but designed for the T5 architecture.

1.1: Encoder Block — Implement T5 encoder block with bidirectional self-attention, feed-forward network, layer normalization, and residual connections

1.2: Encoder Stack — Stack multiple encoder blocks with positional encoding and token embeddings

1.3: Encoder Testing — Test encoder implementation and verify bidirectional attention works correctly

```

1 # Helper function: Positional Encoding
2 class PositionalEncoding(nn.Module):
3     def __init__(self, d_model, max_len=5000):
4         super().__init__()
5         position = torch.arange(max_len).unsqueeze(1)
6         div_term = torch.exp(torch.arange(0, d_model, 2) * -(math.log(10000.0) / d_model))
7
8         pe = torch.zeros(max_len, d_model)
9         pe[:, 0::2] = torch.sin(position * div_term)
10        pe[:, 1::2] = torch.cos(position * div_term)
11
12        self.register_buffer('pe', pe)
13
14    def forward(self, x):
15        # x shape: (batch_size, seq_len, d_model)
16        return x + self.pe[:x.size(1), :].unsqueeze(0)

```

```

1 class FeedForward(nn.Module):
2     def __init__(self, n_embd, hidden_size, dropout=0.1):
3         super().__init__()
4         self.net = nn.Sequential(
5             nn.Linear(n_embd, hidden_size),
6             nn.GELU(),
7             nn.Dropout(dropout),
8             nn.Linear(hidden_size, n_embd),
9             nn.Dropout(dropout),
10        )
11    def forward(self, x): return self.net(x)

```

```

1 # Helper function: Scaled Dot-Product Attention
2 def scaled_dot_product_attention(query, key, value, mask=None, dropout=None):

```

```

3     """
4     Compute scaled dot-product attention.
5
6     Args:
7         query: [batch_size, seq_len, d_k] or [batch_size, n_heads, seq_len, d_k]
8         key: [batch_size, seq_len, d_k] or [batch_size, n_heads, seq_len, d_k]
9         value: [batch_size, seq_len, d_v] or [batch_size, n_heads, seq_len, d_v]
10        mask: [batch_size, seq_len, seq_len] or None (0s for masked positions)
11        dropout: Dropout layer or None
12    Returns:
13        output: [batch_size, seq_len, d_v] or [batch_size, n_heads, seq_len, d_v]
14        attention_weights: [batch_size, seq_len, seq_len] or [batch_size, n_heads, seq_len, seq_len]
15    """
16    d_k = query.size(-1)
17
18    # Compute attention scores: Q @ K^T / sqrt(d_k)
19    scores = torch.matmul(query, key.transpose(-2, -1)) / math.sqrt(d_k)
20
21    # Apply mask if provided (set masked positions to -inf before softmax)
22    if mask is not None:
23        scores = scores.masked_fill(mask == 0, -1e9)
24
25    # Apply softmax to get attention weights
26    attention_weights = F.softmax(scores, dim=-1)
27
28    # Apply dropout if provided
29    if dropout is not None:
30        attention_weights = dropout(attention_weights)
31
32    # Multiply attention weights by values
33    output = torch.matmul(attention_weights, value)
34
35    return output, attention_weights

```

```

1 class MultiHeadAttention(nn.Module):
2     def __init__(self, n_embd, n_head, dropout=0.1):
3         super().__init__()
4         assert n_embd % n_head == 0
5         self.n_head = n_head
6         self.head_dim = n_embd // n_head
7         self.scale = self.head_dim ** -0.5
8         self.query = nn.Linear(n_embd, n_embd)
9         self.key = nn.Linear(n_embd, n_embd)
10        self.value = nn.Linear(n_embd, n_embd)
11        self.proj = nn.Linear(n_embd, n_embd)
12        self.dropout = nn.Dropout(dropout)
13
14    def forward(self, q, k, v, mask=None):
15        B, T, C = q.shape
16        q = self.query(q).view(B, -1, self.n_head, self.head_dim).transpose(1, 2)
17        k = self.key(k).view(B, -1, self.n_head, self.head_dim).transpose(1, 2)
18        v = self.value(v).view(B, -1, self.n_head, self.head_dim).transpose(1, 2)
19        scores = (q @ k.transpose(-2, -1)) * self.scale
20        if mask is not None:
21            scores = scores.masked_fill(mask == 0, float('-inf'))
22        attn = self.dropout(F.softmax(scores, dim=-1))
23        out = (attn @ v).transpose(1, 2).contiguous().view(B, -1, C)
24        return self.proj(out), attn

```

✓ Task 1.1 Implement T5 encoder block

Implement a T5 encoder block with:

- Bidirectional multi-head self-attention (all positions can attend to all positions)
- Feed-forward network
- Layer normalization (pre-norm architecture: normalize before sub-layers)
- Residual connections around each sub-layer

The encoder block should follow this structure:

1. Self-attention: $x = x + \text{dropout}(\text{attention}(\text{layer_norm}(x)))$
2. Feed-forward: $x = x + \text{dropout}(\text{ffn}(\text{layer_norm}(x)))$

```

1 class T5EncoderBlock(nn.Module):
2     """T5 Encoder Block with bidirectional self-attention."""
3     def __init__(self, d_model, n_heads, d_ff, dropout=0.1):
4         super().__init__()
5         ## YOUR_CODE_STARTS_HERE

```

```

6     # TODO: Initialize:
7     # - Multi-head self-attention (bidirectional, no masking)
8     # - Feed-forward network
9     # - Two layer normalization layers (one for attention, one for FFN)
10    # - Dropout layer
11
12    self.self_attention = MultiHeadAttention(d_model, n_heads, dropout)
13    self.feed_forward = FeedForward(d_model, d_ff, dropout)
14    self.norm_1 = nn.LayerNorm(d_model)
15    self.norm_2 = nn.LayerNorm(d_model)
16    self.dropout = nn.Dropout(dropout)
17
18    ## YOUR_CODE_ENDS_HERE
19
20    def forward(self, x, mask=None):
21        ## YOUR_CODE_STARTS_HERE
22        # TODO: Implement encoder block with residual connections (pre-norm)
23        # 1. Self-attention: x = x + dropout(attention(layer_norm(x)))
24        # 2. Feed-forward: x = x + dropout(ffn(layer_norm(x)))
25        # Note: Apply layer norm BEFORE the sub-layer (pre-norm architecture)
26
27        attention, _ = self.self_attention(self.norm_1(x), self.norm_1(x), self.norm_1(x), mask = mask)
28        x = x + self.dropout(attention)
29
30        feed_forward = self.feed_forward(self.norm_2(x))
31        x = x + self.dropout(feed_forward)
32
33
34        ## YOUR_CODE_ENDS_HERE
35
36    return x

```

```

1 # Test your implementation
2 encoder_block = TSEncoderBlock(d_model=512, n_heads=8, d_ff=2048)
3 x = torch.randn(2, 10, 512)
4 output = encoder_block(x)
5 print(f"Encoder block output shape: {output.shape}")
6 ## RESULT_CHECKING_POINT -> torch.Size([2, 10, 512])

```

```
Encoder block output shape: torch.Size([2, 10, 512])
```

✓ Task 1.2 Build encoder stack

```

1 class TSEncoder(nn.Module):
2     """T5 Encoder: Stack of encoder blocks with embeddings and positional encoding."""
3     def __init__(self, vocab_size, d_model, n_heads, n_layers, d_ff, max_len=5000, dropout=0.1):
4         super().__init__()
5         ## YOUR_CODE_STARTS_HERE
6         # TODO: Initialize:
7         # - Token embedding layer
8         # - Positional encoding
9         # - Stack of encoder blocks (n_layers)
10        # - Final layer normalization (optional, but common in T5)
11
12        self.token_embedding = nn.Embedding(vocab_size, d_model)
13        self.pos_embedding = PositionalEncoding(d_model, max_len)
14        self.dropout = nn.Dropout(dropout)
15        self.blocks = nn.ModuleList([TSEncoderBlock(d_model, n_heads, d_ff, dropout) for _ in range(n_layers)])
16        self.norm = nn.LayerNorm(d_model)
17
18        ## YOUR_CODE_ENDS_HERE
19
20    def forward(self, x, mask=None):
21        ## YOUR_CODE_STARTS_HERE
22        # TODO: Implement encoder forward pass
23        # 1. Token embeddings
24        # 2. Add positional encoding
25        # 3. Apply dropout
26        # 4. Pass through encoder blocks
27        # 5. Apply final layer norm
28
29        tok_embed = self.token_embedding(x)
30        x = self.pos_embedding(tok_embed)
31        x = self.dropout(x)
32        for block in self.blocks:
33            x = block(x, mask)
34        x = self.norm(x)
35        ## YOUR_CODE_ENDS_HERE
36
37    return x

```

```

38
39 # Test your implementation
40 encoder = T5Encoder(vocab_size=1000, d_model=512, n_heads=8, n_layers=6, d_ff=2048)
41 x = torch.randint(0, 1000, (2, 10))
42 output = encoder(x)
43 print(f"Encoder output shape: {output.shape}")
44 ## RESULT_CHECKING_POINT -> torch.Size([2, 10, 512])

```

Encoder output shape: torch.Size([2, 10, 512])

Task 1.3 Test encoder implementation

```

1 def test_encoder(encoder, sample_text, max_len=50):
2     """Test encoder and visualize attention patterns."""
3     encoder.eval()
4
5     # Encode text → ensure tensors go to the correct device
6     encoded = torch.tensor(encode(sample_text[:max_len]), device=device).unsqueeze(0)
7
8     # Forward pass
9     with torch.no_grad():
10         output = encoder(encoded)
11
12     print(f"Input text: {sample_text[:max_len]}")
13     print(f"Input shape: {encoded.shape}")
14     print(f"Encoder output shape: {output.shape}")
15     print(f"Encoder output mean: {output.mean().item():.4f}")
16     print(f"Encoder output std: {output.std().item():.4f}")
17
18     return output

```

Task 2: T5 Decoder with Cross-Attention (7 points)

Build the decoder stack with causal self-attention and cross-attention to encoder outputs. The decoder uses masked self-attention (causal) for autoregressive generation and cross-attention to attend to the encoder's output.

2.1: Cross-Attention Mechanism — Implement cross-attention where queries come from decoder states, and keys/values come from encoder outputs

2.2: T5 Decoder Block — Implement decoder block with masked self-attention, cross-attention, and feed-forward network

2.3: Decoder Stack — Stack multiple decoder blocks with positional encoding and token embeddings

Task 2.1 Implement cross-attention mechanism

```

1 class CrossAttentionHead(nn.Module):
2     """Single head of cross-attention (decoder attends to encoder)."""
3     def __init__(self, head_size, n_embd, dropout):
4         super().__init__()
5         ## YOUR_CODE_STARTS_HERE
6         # Query comes from decoder, Key and Value come from encoder
7
8         self.query = nn.Linear(n_embd, head_size)
9         self.key = nn.Linear(n_embd, head_size)
10        self.value = nn.Linear(n_embd, head_size)
11
12        self.scale = head_size ** (-0.5) # scale for attention scores
13        self.dropout = nn.Dropout(dropout)
14
15        ## YOUR_CODE_ENDS_HERE
16
17    def forward(self, decoder_states, encoder_states):
18        ## YOUR_CODE_STARTS_HERE
19
20        q = self.query(decoder_states) # only query comes from decoder
21        k = self.key(encoder_states)
22        v = self.value(encoder_states)
23        scores = q @ k.transpose(-2, -1) * self.scale # attention scores
24
25        attention = F.softmax(scores, dim=-1)
26        attention = self.dropout(attention)
27
28        out = torch.matmul(attention, v) # product of attention weights and values
29
30        ## YOUR_CODE_ENDS_HERE
31        return out

```

```

1 class CrossMultiHeadAttention(nn.Module):
2     """Multi-head cross-attention for T5."""
3     def __init__(self, n_head, head_size, n_embd, dropout):
4         super().__init__()
5         ## YOUR_CODE_STARTS_HERE
6         assert n_embd % n_head == 0 # equal split among heads
7         self.n_head = n_head
8         self.head_dim = n_embd // n_head
9
10        # Q, K, V from concatenated heads
11        self.query = nn.Linear(n_embd, n_embd)
12        self.key = nn.Linear(n_embd, n_embd)
13        self.value = nn.Linear(n_embd, n_embd)
14
15        self.proj = nn.Linear(n_embd, n_embd)
16
17        self.scale = self.head_dim ** (-0.5)
18        self.dropout = nn.Dropout(dropout)
19
20        ## YOUR_CODE_ENDS_HERE
21
22    def forward(self, decoder_states, encoder_states):
23        ## YOUR_CODE_STARTS_HERE
24
25        B, T_decoder, _ = decoder_states.shape
26        T_encoder = encoder_states.size(1)
27
28        q = self.query(decoder_states) # only query from decoder
29        k = self.key(encoder_states)
30        v = self.value(encoder_states)
31
32        # reshape to fit multiple heads
33        q = q.view(B, T_decoder, self.n_head, self.head_dim).transpose(1, 2)
34        k = k.view(B, T_encoder, self.n_head, self.head_dim).transpose(1, 2)
35        v = v.view(B, T_encoder, self.n_head, self.head_dim).transpose(1, 2)
36
37        scores = q @ k.transpose(-2,-1) * self.scale
38
39        attention = F.softmax(scores, dim=-1)
40        attention = self.dropout(attention)
41
42        out = torch.matmul(attention, v)
43
44        # reconcatenate heads
45        out = out.transpose(1,2).contiguous().view(B, T_decoder, -1)
46        out = self.proj(out)
47
48        ## YOUR_CODE_ENDS_HERE
49        return out

```

```

1 # Test your implementation
2 cross_attn = CrossMultiHeadAttention(n_head=8, head_size=64, n_embd=512, dropout=0.1)
3 decoder_states = torch.randn(2, 10, 512) # (batch, decoder_seq_len, d_model)
4 encoder_states = torch.randn(2, 15, 512) # (batch, encoder_seq_len, d_model)
5 output = cross_attn(decoder_states, encoder_states)
6 print(f"Cross-attention output shape: {output.shape}")
7 ## RESULT_CHECKING_POINT -> torch.Size([2, 10, 512])

```

Cross-attention output shape: torch.Size([2, 10, 512])

✓ Task 2.2 Implement T5 decoder block

Implement the decoder block with:

- Masked self-attention (causal, for autoregressive generation)
- Cross-attention (decoder attends to encoder)
- Feed-forward network
- Three layer normalization layers (pre-norm)
- Residual connections

```

1 def generate_causal_mask(seq_len):
2     return torch.tril(torch.ones(seq_len, seq_len))

```

```

1 class T5DecoderBlock(nn.Module):
2     def __init__(self, n_embd, n_head, block_size, dropout):
3         super().__init__()
4         head_size = n_embd // n_head
5         ## YOUR_CODE_STARTS_HERE

```

```

6         self.self_attention = MultiHeadAttention(
7             n_head = n_head,
8             n_embd = n_embd,
9             dropout = dropout,
10        )
11        self.cross_attention = CrossMultiHeadAttention(
12            n_head = n_head,
13            head_size = head_size,
14            n_embd = n_embd,
15            dropout = dropout
16        )
17        self.feed_forward = nn.Sequential(
18            nn.Linear(n_embd, 4*n_embd),
19            nn.ReLU(),
20            nn.Linear(4*n_embd, n_embd),
21            nn.Dropout(dropout)
22        )
23        self.dropout = nn.Dropout(dropout)
24
25        self.norm_1 = nn.LayerNorm(n_embd)
26        self.norm_2 = nn.LayerNorm(n_embd)
27        self.norm_3 = nn.LayerNorm(n_embd)
28
29        ## YOUR_CODE_ENDS_HERE
30
31    def forward(self, x, encoder_output):
32        ## YOUR_CODE_STARTS_HERE
33        B, T, _ = x.size()
34        # casual mask for self attention
35        _mask = torch.tril(torch.ones(T, T, device = x.device)).unsqueeze(0).unsqueeze(0)
36
37        attention, _ = self.self_attention(self.norm_1(x), self.norm_1(x), self.norm_1(x), mask = _mask)
38        x = x + self.dropout(attention)
39
40        # decoder attend to encoder in cross-attention
41        x = x + self.cross_attention(self.norm_2(x), encoder_output)
42
43        feed_forward = self.feed_forward(self.norm_3(x))
44        x = x + self.dropout(feed_forward)
45
46        ## YOUR_CODE_ENDS_HERE
47        return x

```

Task 2.3 Build decoder stack

```

1 class T5Decoder(nn.Module):
2     def __init__(self, vocab_size, d_model, n_heads, n_layers, d_ff, block_size, max_len, dropout):
3         super().__init__()
4         ## YOUR_CODE_STARTS_HERE
5
6         self.token_embedding = nn.Embedding(vocab_size, d_model)
7         self.pos_embedding = PositionalEncoding(d_model, block_size)
8         self.dropout = nn.Dropout(dropout)
9         # stack of decoder blocks
10        self.blocks = nn.ModuleList([T5DecoderBlock(d_model, n_heads, block_size, dropout) for _ in range(n_layers)])
11        self.norm = nn.LayerNorm(d_model)
12        self.block_size = block_size
13
14        ## YOUR_CODE_ENDS_HERE
15
16    def forward(self, x, encoder_output):
17        ## YOUR_CODE_STARTS_HERE
18
19        tok_embed = self.token_embedding(x)
20        x = self.pos_embedding(tok_embed)
21        x = self.dropout(x)
22        for block in self.blocks:
23            x = block(x, encoder_output)
24        x = self.norm(x)
25
26        return x
27
28        ## YOUR_CODE_ENDS_HERE

```

```

1 print("Testing T5DecoderBlock...")
2 decoder_block = T5DecoderBlock(n_embd=512, n_head=8, block_size=128, dropout=0.1)
3 x = torch.randn(2, 10, 512)
4 encoder_output = torch.randn(2, 15, 512)

```



```

5 output = decoder_block(x, encoder_output)
6 print(f"Decoder block output shape: {output.shape}")

```

```

Testing T5DecoderBlock...
Decoder block output shape: torch.Size([2, 10, 512])

```

✓ TASK 3: Full T5 Model and Machine Translation (7 points)

Combine encoder and decoder into a complete T5 model and train it on English-German machine translation.

3.1: Complete T5 Model — Combine encoder and decoder stacks with shared token embeddings and language modeling head

3.2: Translation Dataset Preparation — Prepare English-German translation dataset with proper batching and padding

3.3: Training and Testing — Train model on translation task and evaluate performance

Task 3.1 Build complete T5 model

```

1 class T5Model(nn.Module):
2     def __init__(self, vocab_size, d_model, n_heads, n_layers, d_ff, block_size, max_len=5000, dropout=0.1):
3         super().__init__()
4         ## YOUR_CODE_STARTS_HERE
5
6         self.vocab_size = vocab_size
7
8         self.encoder = T5Encoder(      # Encoder
9             vocab_size = vocab_size,
10            d_model = d_model,
11            n_heads = n_heads,
12            n_layers = n_layers,
13            d_ff = d_ff,
14            max_len = max_len,
15            dropout = dropout
16        )
17
18        self.decoder = T5Decoder(      # Decoder
19            vocab_size = vocab_size,
20            d_model = d_model,
21            n_heads = n_heads,
22            n_layers = n_layers,
23            d_ff = d_ff,
24            block_size = block_size,
25            max_len = max_len,
26            dropout = dropout
27        )
28
29        # shared embeddings for both
30        self.shared_token_embeddings = nn.Embedding(vocab_size, d_model)
31        self.encoder.token_embedding = self.shared_token_embeddings
32        self.decoder.token_embedding = self.shared_token_embeddings
33
34        self.lm_head = nn.Linear(d_model, vocab_size, bias = False)
35        self.lm_head.weight = self.shared_token_embeddings.weight
36
37        ## YOUR_CODE_ENDS_HERE
38
39    def forward(self, src, tgt):
40        ## YOUR_CODE_STARTS_HERE
41
42        encoder_output = self.encoder(src) # encode source
43        decoder_states = self.decoder(tgt, encoder_output) # decode based on encoding output
44
45        logits = self.lm_head(decoder_states) # next token prediction
46
47        ## YOUR_CODE_ENDS_HERE
48        return logits

```

✓ Task 3.2 Prepare translation dataset

```

1 def get_translation_batch(split, batch_size=4):
2     """Get a batch of English-German translation pairs."""
3     pairs = train_pairs if split == 'train' else val_pairs
4
5     # Random sampling
6     indices = torch.randint(0, len(pairs), (batch_size,))
7
8     src_batch = []
9     tgt_batch = []

```

```

10
11     for idx in indices:
12         en_text, de_text = pairs[idx]
13         src_ids = encode(en_text)
14         # German: Add SOS at start, EOS at end
15         tgt_ids = [SOS_IDX] + encode(de_text) + [EOS_IDX]
16
17         src_batch.append(torch.tensor(src_ids, dtype=torch.long))
18         tgt_batch.append(torch.tensor(tgt_ids, dtype=torch.long))
19
20     # Pad
21     src_batch = torch.nn.utils.rnn.pad_sequence(src_batch, batch_first=True, padding_value=PAD_IDX)
22     tgt_batch = torch.nn.utils.rnn.pad_sequence(tgt_batch, batch_first=True, padding_value=PAD_IDX)
23
24     # Teacher forcing split
25     tgt_input = tgt_batch[:, :-1] # Input: SOS ... Token
26     tgt_output = tgt_batch[:, 1:] # Target: Token ... EOS
27
28     return src_batch.to(device), tgt_input.to(device), tgt_output.to(device)

```

```

1 BLOCK_SIZE = 512 # Increased from 64 to 512 to handle sentences > 64 chars
2 BATCH_SIZE = 16
3
4 model = T5Model(
5     vocab_size=vocab_size,
6     d_model=128,
7     n_heads=4,
8     n_layers=2,
9     d_ff=512,
10    block_size=BLOCK_SIZE
11 ).to(device)
12
13 print(f"Model initialized with block_size={BLOCK_SIZE}")
14
15 optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3)
16 criterion = nn.CrossEntropyLoss(ignore_index=PAD_IDX)
17
18 # Updated Batch Function with Truncation
19 def get_safe_batch(split, batch_size=4, max_len=BLOCK_SIZE):
20     src_batch, tgt_in, tgt_out = get_translation_batch(split, batch_size)
21
22     # Truncate if they exceed block_size
23     if src_batch.size(1) > max_len:
24         src_batch = src_batch[:, :max_len]
25     if tgt_in.size(1) > max_len:
26         tgt_in = tgt_in[:, :max_len]
27         tgt_out = tgt_out[:, :max_len] # Output must match input length
28
29     return src_batch, tgt_in, tgt_out
30
31 print("Starting training...")
32 model.train()

```

```

Model initialized with block_size=512
Starting training...
T5Model(
  (encoder): T5Encoder(
    (token_embedding): Embedding(113, 128)
    (pos_embedding): PositionalEncoding()
    (dropout): Dropout(p=0.1, inplace=False)
    (blocks): ModuleList(
      (0-1): 2 x T5EncoderBlock(
        (self_attention): MultiHeadAttention(
          (query): Linear(in_features=128, out_features=128, bias=True)
          (key): Linear(in_features=128, out_features=128, bias=True)
          (value): Linear(in_features=128, out_features=128, bias=True)
          (proj): Linear(in_features=128, out_features=128, bias=True)
          (dropout): Dropout(p=0.1, inplace=False)
        )
        (feed_forward): FeedForward(
          (net): Sequential(
            (0): Linear(in_features=128, out_features=512, bias=True)
            (1): GELU(approximate='none')
            (2): Dropout(p=0.1, inplace=False)
            (3): Linear(in_features=512, out_features=128, bias=True)
            (4): Dropout(p=0.1, inplace=False)
          )
        )
      )
    )
    (norm_1): LayerNorm((128,), eps=1e-05, elementwise_affine=True)
    (norm_2): LayerNorm((128,), eps=1e-05, elementwise_affine=True)
    (dropout): Dropout(p=0.1, inplace=False)
  )
)
(norm): LayerNorm((128,), eps=1e-05, elementwise_affine=True)

```

```

)
(decoder): T5Decoder(
  (token_embedding): Embedding(113, 128)
  (pos_embedding): PositionalEncoding()
  (dropout): Dropout(p=0.1, inplace=False)
  (blocks): ModuleList(
    (0-1): 2 x T5DecoderBlock(
      (self_attention): MultiHeadAttention(
        (query): Linear(in_features=128, out_features=128, bias=True)
        (key): Linear(in_features=128, out_features=128, bias=True)
        (value): Linear(in_features=128, out_features=128, bias=True)
        (proj): Linear(in_features=128, out_features=128, bias=True)
        (dropout): Dropout(p=0.1, inplace=False)
      )
      (cross_attention): CrossMultiHeadAttention(
        (query): Linear(in_features=128, out_features=128, bias=True)
        (key): Linear(in_features=128, out_features=128, bias=True)
        (value): Linear(in_features=128, out_features=128, bias=True)
        (proj): Linear(in_features=128, out_features=128, bias=True)
        (dropout): Dropout(p=0.1, inplace=False)
      )
      (feed_forward): Sequential(
        (0): Linear(in_features=128, out_features=512, bias=True)
        (1): ReLU()
        (2): Linear(in_features=512, out_features=128, bias=True)
        (3): Dropout(p=0.1, inplace=False)
      )
    )
  )
)

```

Task 3.3 Train and test translation model

```

1 PRINT_EVERY = 50
2
3 for epoch in range(5): # or more epochs later
4     for step in range(len(train_pairs)//BATCH_SIZE):
5         src, tgt_in, tgt_out = get_safe_batch('train', batch_size=BATCH_SIZE)
6
7         logits = model(src, tgt_in)
8         loss = criterion(logits.reshape(-1, vocab_size), tgt_out.reshape(-1))
9
10        optimizer.zero_grad()
11        loss.backward()
12        optimizer.step()
13
14        # Print every few iterations
15        if step % PRINT_EVERY == 0:
16            print(f"Epoch {epoch} | Step {step}: Loss = {loss.item():.4f}")
17
18 print("✓ Training completed successfully!")
19

```

```

Epoch 0 | Step 0: Loss = 90.7166
Epoch 0 | Step 50: Loss = 4.9869
Epoch 0 | Step 100: Loss = 3.4599
Epoch 0 | Step 150: Loss = 3.0678
Epoch 0 | Step 200: Loss = 2.8170
Epoch 0 | Step 250: Loss = 2.7543
Epoch 0 | Step 300: Loss = 2.6322
Epoch 0 | Step 350: Loss = 2.6316
Epoch 0 | Step 400: Loss = 2.5281
Epoch 0 | Step 450: Loss = 2.5944
Epoch 0 | Step 500: Loss = 2.5094
Epoch 0 | Step 550: Loss = 2.5497
Epoch 0 | Step 600: Loss = 2.4533
Epoch 0 | Step 650: Loss = 2.5393
Epoch 0 | Step 700: Loss = 2.4462
Epoch 0 | Step 750: Loss = 2.5085
Epoch 0 | Step 800: Loss = 2.4955
Epoch 0 | Step 850: Loss = 2.4701
Epoch 0 | Step 900: Loss = 2.5037
Epoch 0 | Step 950: Loss = 2.4439
Epoch 0 | Step 1000: Loss = 2.3893
Epoch 0 | Step 1050: Loss = 2.5087
Epoch 0 | Step 1100: Loss = 2.4971
Epoch 0 | Step 1150: Loss = 2.4654
Epoch 0 | Step 1200: Loss = 2.4127
Epoch 0 | Step 1250: Loss = 2.4322
Epoch 0 | Step 1300: Loss = 2.4713
Epoch 0 | Step 1350: Loss = 2.5853
Epoch 0 | Step 1400: Loss = 2.4151
Epoch 0 | Step 1450: Loss = 2.4202
Epoch 0 | Step 1500: Loss = 2.4623
Epoch 0 | Step 1550: Loss = 2.5337
Epoch 0 | Step 1600: Loss = 2.4413
Epoch 0 | Step 1650: Loss = 2.4022
Epoch 0 | Step 1700: Loss = 2.4058
Epoch 0 | Step 1750: Loss = 2.3304
Epoch 0 | Step 1800: Loss = 2.4005

```

```

Epoch 0 | Step 1850: Loss = 2.4214
Epoch 0 | Step 1900: Loss = 2.3663
Epoch 0 | Step 1950: Loss = 2.3381
Epoch 0 | Step 2000: Loss = 2.4006
Epoch 0 | Step 2050: Loss = 2.3965
Epoch 0 | Step 2100: Loss = 2.4023
Epoch 0 | Step 2150: Loss = 2.3574
Epoch 0 | Step 2200: Loss = 2.3707
Epoch 0 | Step 2250: Loss = 2.3921
Epoch 0 | Step 2300: Loss = 2.3745
Epoch 0 | Step 2350: Loss = 2.2917
Epoch 0 | Step 2400: Loss = 2.3370
Epoch 0 | Step 2450: Loss = 2.2878
Epoch 1 | Step 0: Loss = 2.3080
Epoch 1 | Step 50: Loss = 2.2520
Epoch 1 | Step 100: Loss = 2.2469
Epoch 1 | Step 150: Loss = 2.1913
Epoch 1 | Step 200: Loss = 2.3027
Epoch 1 | Step 250: Loss = 2.2676
Epoch 1 | Step 300: Loss = 2.3106
Epoch 1 | Step 350: Loss = 2.2111

```

```

1 # Set model to evaluation mode (disables dropout)
2 model.eval()
3
4 def translate_sentence(sentence, model, max_length=100):
5     """
6     Translates an English sentence to German using the trained T5Model.
7     Uses greedy decoding.
8     """
9     # 1. Prepare Source (English)
10    # Encode and add batch dimension [1, seq_len]
11    src_ids = encode(sentence)
12    src_tensor = torch.tensor(src_ids, dtype=torch.long).unsqueeze(0).to(device)
13
14    # 2. Encoder Pass
15    # We encode the source sequence once
16    with torch.no_grad():
17        encoder_output = model.encoder(src_tensor)
18
19    # 3. Decoder Loop (Autoregressive)
20    # Start with the Start-Of-Sequence token
21    decoder_input = torch.tensor([[SOS_IDX]], dtype=torch.long).to(device)
22
23    generated_tokens = []
24
25    for _ in range(max_length):
26        with torch.no_grad():
27            # Pass current sequence and encoder output to decoder
28            # decoder_input shape: [1, current_seq_len]
29            decoder_output = model.decoder(decoder_input, encoder_output)
30
31            # Project to vocabulary
32            logits = model.lm_head(decoder_output)
33
34            # Get the token with highest probability for the last position
35            # logits shape: [1, current_seq_len, vocab_size]
36            next_token_logits = logits[:, -1, :]
37            next_token_id = torch.argmax(next_token_logits, dim=-1).item()
38
39            # Stop if End-Of-Sequence token is generated
40            if next_token_id == EOS_IDX:
41                break
42
43            generated_tokens.append(next_token_id)
44
45            # Append prediction to decoder input for next step
46            next_token_tensor = torch.tensor([[next_token_id]], dtype=torch.long).to(device)
47            decoder_input = torch.cat([decoder_input, next_token_tensor], dim=1)
48
49    # 4. Decode to string
50    translated_text = decode(generated_tokens)
51    return translated_text
52
53 # --- Test Translations ---
54 print("--- Translation Demo ---")
55 test_sentences = [
56     "Hello",
57     "Good morning",
58     "I love you",
59     "Where is the bathroom?",
60     "This is difficult"
61 ]
62

```

```
63 for s in test_sentences:
64     trans = translate_sentence(s, model)
65     print(f"EN: {s[:25]} -> DE: {trans}")
66
67 # Try a custom one
68 custom = "I am hungry"
69 print(f"\nCustom: {custom} -> {translate_sentence(custom, model)}")
```

```
--- Translation Demo ---
EN: Hello                -> DE: Deee insmmmlieeellleen undeeeeeeeuuueeeeeeeuuueeeeeeuuneeuuuueeeeeuuueeeeeuuuung
EN: Good morning          -> DE: Die Ausssssuchungsnicht der Ausssusssssuchungen der Aussssuussssssuchungsnisseondern und d
EN: I love you            -> DE: Ich mmmnenke, werdeeeinnungssichtlich nicht geoneeeeeeeinauseeeuuungsnicht werden, de
EN: Where is the bathroom? -> DE: Wir ssind schween wir sind schweendigeeeeeeeeeeeeeeeinein und deeinealle und schiede
EN: This is difficult     -> DE: Diess sind ssseisssontischeidungsssorgeeit deee in deeeinsationalisiein ssteetungssnd

Custom: I am hungry -> Ich mmmin mich mich mmmin deer Schufendee ich eineemminsam Ich aus den Sie und der einealle Anschmit
```

BONUS (2 points): Solve any task with an LLM

Goal.

Pick **one** of the homework tasks (Task 1, Task 2, or Task 3) and solve it using an **LLM**. Provide the **LLM** with the **task description** and any **starter/prerequisite code** it depends on, ask it to generate a complete **code solution** first, then run that **generated code** here in the code notebook yourself. Finally, document what you did and **compare** the LLM's result to your own pipeline.

What to deliver below.

1. **LLM used** (name + version, e.g., “Llama-3-8B-Instruct”, “GPT-x”, “Claude-x”, “Mistral-x”, etc.).
2. **Prompt(s)** you used.
3. **LLM output** — copy and paste the generated code.
4. **Comparison** to your solution: what matches or differs (quantitative or qualitative).
5. **Reflection**: what the LLM was **good at** vs **bad at**, what it got **right** vs **wrong**.

No code required. You do **not** need to run, share, or submit any code used for the LLM generation. Provide only the deliverables listed above. You may use any LLMs through any interface (API, web UI, local inference).

✓ Solution

1. **LLM used:** GPT-5
2. **Prompts:** I attached the SetUp, the Dataset Preparation and the helper functions: PositionalEncoding, FeedForward, scaled_dot_product_attention and MultiHeadAttention. I asked GPT-5 to use the helper code in order to solve the first task providing the description.
3. **LLM Output:**

```

1 class RMSNorm(nn.Module):
2     def __init__(self, dim, eps=1e-6):
3         super().__init__()
4         self.eps = eps
5         self.weight = nn.Parameter(torch.ones(dim))
6
7     def forward(self, x):
8         # root mean square
9         rms = x.pow(2).mean(dim=-1, keepdim=True).add(self.eps).sqrt()
10        x = x / rms
11        return self.weight * x

```

```

1 class T5EncoderBlock(nn.Module):
2     def __init__(self, n_embd, n_head, ff_hidden, dropout=0.1):
3         super().__init__()
4
5         # --- Layer norms (RMSNorm for real T5) ---
6         self.norm1 = RMSNorm(n_embd)
7         self.norm2 = RMSNorm(n_embd)
8
9         # --- Attention ---
10        self.self_attn = MultiHeadAttention(n_embd, n_head, dropout=dropout)
11
12        # --- Feedforward ---
13        self.ff = FeedForward(n_embd, ff_hidden, dropout=dropout)
14
15        self.dropout = nn.Dropout(dropout)
16
17        def forward(self, x, mask=None):

```

```

18     # ----- Self-attention (pre-norm) -----
19     h = self.norm1(x)
20     attn_out, _ = self.self_attn(h, h, h, mask=mask)
21     x = x + self.dropout(attn_out)
22
23     # ----- Feed-forward (pre-norm) -----
24     h = self.norm2(x)
25     ff_out = self.ff(h)
26     x = x + self.dropout(ff_out)
27
28     return x

```

```

1 class TSEncoder(nn.Module):
2     def __init__(
3         self,
4         vocab_size,
5         n_embd=256,
6         n_head=4,
7         ff_hidden=1024,
8         num_layers=6,
9         max_len=512,
10        dropout=0.1
11    ):
12        super().__init__()
13
14        self.token_emb = nn.Embedding(vocab_size, n_embd)
15        self.pos_enc = PositionalEncoding(n_embd, max_len)
16
17        self.layers = nn.ModuleList([
18            TSEncoderBlock(n_embd, n_head, ff_hidden, dropout)
19            for _ in range(num_layers)
20        ])
21
22        self.norm = RMSNorm(n_embd)
23
24    def forward(self, x, mask=None):
25        # x: (batch, seq)
26        x = self.token_emb(x)
27        x = self.pos_enc(x)
28
29        for layer in self.layers:
30            x = layer(x, mask)
31
32        return self.norm(x)

```

```

1 def test_encoder():
2     batch = 2
3     seqlen = 20
4     vocab = vocab_size
5     n_embd = 128
6     n_head = 4
7     ff_hidden = 256
8     layers = 3
9
10    encoder = TSEncoder(
11        vocab_size=vocab,
12        n_embd=n_embd,
13        n_head=n_head,
14        ff_hidden=ff_hidden,
15        num_layers=layers,
16        max_len=512,
17        dropout=0.1
18    ).to(device)
19
20    # Random input
21    x = torch.randint(0, vocab, (batch, seqlen)).to(device)
22
23    # Mask: 1 = keep, 0 = pad
24    mask = (x != PAD_IDX).unsqueeze(1).unsqueeze(2)
25
26    print("Input shape:", x.shape)
27    print("Mask shape:", mask.shape)
28
29    out = encoder(x, mask)
30    print("Output shape:", out.shape)
31
32    # Check NaNs
33    assert not torch.isnan(out).any(), "NaNs detected in encoder output!"
34
35    # Gradient test
36    out.sum().backward()

```

```
37     print("Backprop OK.")
38
39     print("✓ Encoder test passed!")
```

```
1 test_encoder()
```

```
Input shape: torch.Size([2, 20])
Mask shape: torch.Size([2, 1, 1, 20])
Output shape: torch.Size([2, 20, 128])
Backprop OK.
✓ Encoder test passed!
```

4. **Comparison:** The main difference noticed in the LLM implementation is the use of RMS norm arguing that the T5 does not use Layer norm but instead the root-mean-square.

The T5 Encoder Block follows the same logic as implemented originally, using Multi Head Attention and implementing Self